

1. Introdução e Objetivos

Após a especificação do modelo de dados e a criação das rotas na API através do LoopBack 4, tornou-se necessária a criação de uma interface de utilizador (Frontend) que permitisse à receção e à administração gerir o sistema de forma visual e intuitiva.

Para cumprir os requisitos técnicos da unidade curricular de **Sistemas de Informação e Base de Dados**, recorreremos ao *framework* **React-Admin**. O objetivo deste documento é relatar o processo de desenvolvimento e os desafios superados para pôr a interface a comunicar corretamente com o nosso *backend*.

2. Estrutura de Componentes e Interface Visual

Para garantir uma experiência de utilização profissional e organizada, o desenvolvimento focou-se em quatro pilares visuais e funcionais:

- **Dashboard Inicial:** Criámos um ecrã de boas-vindas utilizando componentes da biblioteca Material UI (Card, CardHeader e CardContent). Este ecrã serve de ponto de entrada no sistema e contextualiza as operações da clínica.
- **Menu Lateral com Ícones Customizados:** Para facilitar a navegação rápida, integrámos a biblioteca @mui/icons-material. Cada uma das cinco entidades centrais do nosso modelo de dados recebeu um ícone representativo e específico (ex: LocalHospitalIcon para as Especialidades e PeopleIcon para os Pacientes).
- **Formulários de Criação e Edição:** Substituímos o comportamento padrão de clique na linha (rowClick="edit") por botões explícitos de ação (<EditButton />) no final de cada tabela. Cada entidade foi mapeada com os seus campos exatos (textos e números) usando inputs adequados, garantindo que o ID fica desativado (disabled) durante a edição para proteger a integridade dos dados.
- **Filtros Visuais de Pesquisa:** Seguindo a estrutura metodológica lecionada nesta unidade curricular, criámos componentes de filtragem (<Filter>) com caixas de texto ativas por padrão (alwaysOn). Estes filtros foram acoplados a todas as listagens para permitir uma futura expansão de pesquisas na base de dados.

3. Adaptação do Motor de Dados (dataProvider)

O maior desafio técnico do projeto prendeu-se com o "contrato" de comunicação entre o ecossistema do React-Admin e o comportamento das rotas geradas pelo LoopBack 4. O conector genérico inicial (ra-data-loopback4) apresentou falhas ao tentar mapear operações individuais e chaves primárias.

Para contornar esta limitação sem alterar o comportamento do servidor, desenvolvemos um dataProvider customizado no ficheiro App.tsx que expande o conector base através de mecanismos de tratamento de erros (try/catch):

1. **getList:** Garante o carregamento das tabelas mesmo perante divergências na paginação, assegurando que o atributo de identificação de cada registo (seja id ou _id) é corretamente interpretado pelo React-Admin.
2. **getOne:** Corrige a falha "Element does not exist" que era um erro que estava a persistir, mapeando o ID diretamente no caminho do pedido HTTP (/\${resource}/\${params.id}), permitindo ao formulário ler os dados corretos de um único paciente ou médico.
3. **update e create:** Implementam os métodos HTTP **PUT** e **POST**, respetivamente. Através do envio de cabeçalhos em formato JSON, o sistema passou a ser capaz de injetar novos dados na API ou atualizar o estado de consultas e registos já existentes de forma imediata.